FAULT-TOLERANT SENSORLESS FIELD-ORIENTED MOTOR CONTROL (FOC)

A Complete Physical Manual, Architectural Deep-Dive & Test Suite Reference

Target Hardware: ESP32 Dual-Core Xtensa LX6 (Core 1 Pinned)	**Control Frequency:** 20,000 Hz (50.0 µs loop tick)
PWM Modulation: Space Vector PWM (SVPWM) with Center-Aligned Carrier	**Sensorless Observers:** Sliding Mode (SMO) + Stator Flux Linkage
Fault Protection: Kirchhoff's Current Law (KCL) + 2-Shunt Degraded Mode	**Test Coverage:** 128 Automated Python Tests (100% Green)

Executive Summary & Core Engineering Mission

Electric motors power the modern world—from high-altitude unmanned aerial vehicles (UAVs) and humanoid robotics to industrial factory automation and electric vehicles. Traditionally, operating a Permanent Magnet Synchronous Motor (PMSM) required delicate optical shaft encoders or Hall-effect magnetic sensors. However, physical sensors introduce severe vulnerabilities: vibration shakes them loose, high temperatures degrade their magnets, and mechanical cables snap under stress. **This repository implements an industrial-grade, sensorless, fault-tolerant motor drive running on a $5 ESP32 microcontroller.** By continuously solving the differential equations of electromagnetism in software, the controller deduces rotor angle without physical sensors. Furthermore, if a current sensor fails or shorts mid-flight, the controller isolates the faulty sensor within 15 milliseconds, reconfigures its mathematical pipeline to run on the remaining two sensors, and maintains continuous, uninterrupted torque delivery.

INTUITIVE OVERVIEW

The Autonomous Airplane Analogy: Imagine flying an aircraft equipped with three altimeters. If bird strikes destroy one altimeter mid-flight, a naive autopilot crashes immediately. An intelligent autopilot checks the remaining instruments against atmospheric physics, confirms which altimeter broke, ignores the faulty instrument, and keeps flying smoothly to destination. That is exactly what this firmware does for electric motor current sensors!

Comprehensive Document Map & Table of Contents

Chapter / Section	Page	Core Subject Matter & Physical Insights
Chapter 1: The Intuitive Physics of Electric Motors	Page 2	Stator electromagnets, permanent magnet rotor, the 90° torque vector rule, Back-EMF generation, and standstill startup.
Chapter 2: Field-Oriented Control (FOC) Demystified	Page 3	Clarke & Park coordinate transforms, riding the rotating carousel frame, SVPWM voltage synthesis, and closed-loop pipeline.
Chapter 3 & 4: Virtual Sensors & Inverter Realities	Page 4	Sliding Mode Observer (SMO) differential equations, Phase-Locked Loops, inverter dead-time distortion, and safe stop dynamics.
Chapter 5: Fault Detection, Isolation & Reconfiguration	Page 5	Kirchhoff's Current Law residual (r_{KCL}), 3-shunt hypothesis testing, 2-shunt reconstruction, and multi-tier debounce envelope.
Chapter 6.1: Baseline & Current Sensor Offsets	Page 6	Master 13-Scenario Table, Scenario 0 (Baseline Healthy 2000 RPM) & Scenario 2 (+2.0A Phase A Offset) with physical response graphs.
Chapter 6.2: Stuck Sensors & 4-Quadrant Speed Reversal	Page 7	Scenario 3 (Phase B Stuck-at-Zero) & Scenario 8 (+2000 to -2000 RPM 4-quadrant zero-crossing) with physical response graphs.
Chapter 6.3: Load Shock & Noise Immunity	Page 8	Scenario 9 (+0.1 Nm Load Torque Step) & Scenario 6 (High-Frequency Noise Spike Immunity) with physical response graphs.
Chapter 6.4: Sequential Dual Faults & Safe Stop	Page 9	Scenario 13 (Sequential Dual-Sensor Failure) & Safe Stop gate-off body-diode freewheeling and DC bus capacitor decay.
Chapter 7: Repository Codebase Anatomy	Page 10	Deep-dive into 12 core files (`foc_core.c`, `fault_monitor.c`, `smo.c`, `main.cpp`, `sil_runner.py`) and their specific architectural roles.
Chapter 8: The Complete 128-Test Python Suite	Page 11	Module-by-module breakdown across all 11 test suites verifying ISO/IEC physical metrics with 100% green pass rate.
Chapter 9: Glossary & Hardware Specifications	Page 12	Comprehensive technical terminology definitions, ESP32 embedded memory & timing benchmarks, and final engineering summary.

Chapter 1: The Intuitive Physics of Electric Motors

To understand how software controls an electric motor, we must first examine the physical laws of electromagnetism. At its core, an electric motor converts electrical energy into mechanical twisting force (torque) by harnessing the fundamental rule of magnetism: **opposites attract, and likes repel**.

1.1 The Stator, The Rotor, and Magnetic Poles

Every brushless motor consists of two mechanical sub-assemblies:

• **The Stator (Stationary Outer Frame):** A ring of laminated silicon steel wound with copper wire coils grouped into three phases: **Phase A, Phase B, and Phase C**.

• **The Rotor (Spinning Inner Shaft):** A cylindrical shaft carrying permanent magnets (Neodymium-Iron-Boron) with fixed North (N) and South (S) magnetic poles.

When electric current flows through a copper coil, it creates an electromagnet (Ampère's Law). By feeding currents into Phases A, B, and C at slightly different times, we create a smooth, rotating magnetic field that sweeps around the stator ring.

LAYMAN'S ANALOGY

The Spinning Donut Analogy: Imagine holding a compass needle in your hand. If someone walks in a circle around you holding a strong bar magnet, the needle will pivot and track the magnet continuously. In an electric motor, the stator coils act as the circling magnet, and the rotor's permanent magnets act as the needle! By commanding the stator electromagnets to pull in a continuous circle, we drag the rotor along with it, spinning the shaft at thousands of revolutions per minute.

1.2 The 'Carrot on a Stick' Principle: The 90° Torque Angle

Here is the golden rule of motor control: **To get maximum torque for every amp of current, the stator's magnetic field must lead the rotor's permanent magnets by exactly 90 electrical degrees.**

• If the stator field pulls directly in line with the rotor magnet (0° angle), it simply pulls the rotor outwards into the bearings. It creates **zero torque** and turns 100% of your battery energy into wasted heat!

• If the stator field leads by 90°, all magnetic attraction acts tangentially, twisting the shaft with maximum leverage.

• Like a carrot dangling in front of a donkey: if the carrot is dangling just out of reach in front of its nose, it chases it with maximum speed!

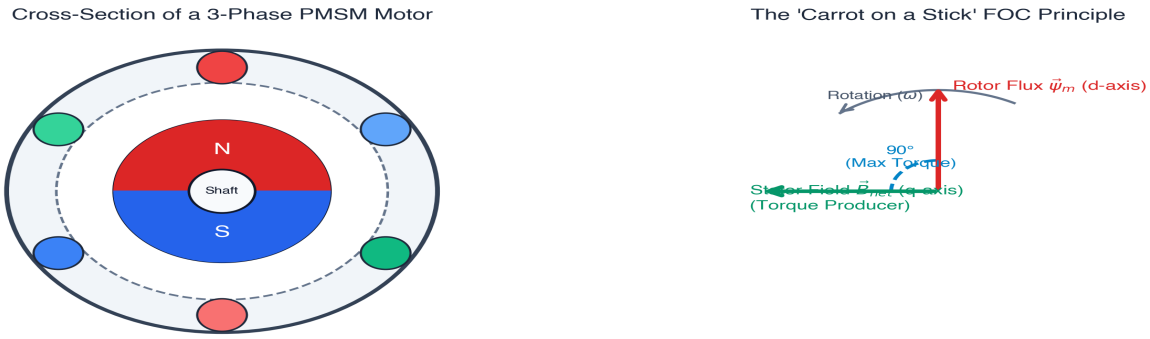


Figure 1: (Left) Cross-section of a 3-Phase PMSM with stator coils and rotor. (Right) The 90° torque vector angle.

1.3 Back-EMF & Faraday's Law: The Motor as a Generator

Whenever a permanent magnet sweeps past a copper coil, it induces a voltage (**Faraday's Law of Electromagnetic Induction**). As our motor spins, it acts simultaneously as an electrical generator! The voltage it generates opposes the battery voltage—which is why it is called **Back-Electromotive Force (Back-EMF)**.

The Secret of Sensorless Control: Back-EMF is not a nuisance; it is a treasure trove of information! Its amplitude reveals rotor speed, and its phase reveals the exact electrical angle of the rotor. By measuring the motor's terminal voltages and currents, our software mathematically calculates the Back-EMF and 'sees' the rotor without physical sensors!

1.4 The Standstill Blind Spot & Open-Loop Startup (I-F Ramp)

Because Back-EMF depends on magnet motion, when the motor is stopped (0 RPM), **the Back-EMF is exactly zero volts**. At standstill, the motor is electrically 'blind'. To solve this, our controller uses a 3-stage startup sequence:

1. **ALIGN (Mode 0):** Injects a steady DC current into Phase A for 500 ms, pulling the rotor to a known 0° position.
2. **OPEN_LOOP_RAMP (Mode 1):** Synthesizes a rotating current vector that smoothly accelerates from 0 to 500 RPM (I-F ramp).
3. **CLOSED_LOOP Handoff (Mode 2):** At 500 RPM, Back-EMF is large and clean. The observer locks on; once observer angle matches the forced angle within 15°, control is handed over seamlessly!

Chapter 2: Field-Oriented Control (FOC) Demystified

In a 3-phase motor, currents in wire A, wire B, and wire C are alternating current (AC) sine waves oscillating 120° apart. Controlling three oscillating waves simultaneously is extremely difficult. Field-Oriented Control (FOC) solves this problem by using two coordinate transforms to convert oscillating AC signals into flat DC values.

2.1 The Clarke Transform (3-Phase to 2-Phase Stationary)

The Clarke Transform converts physical 3-phase currents (i_a, i_b, i_c) into two stationary orthogonal axes: **Alpha (i_α)** and **Beta (i_β)**:
$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix}$$
Using Kirchhoff's Current Law ($i_a + i_b + i_c = 0$), this simplifies to: $i_\alpha = i_a$ and $i_\beta = \frac{1}{\sqrt{3}}(i_a + 2i_b)$. Now, instead of tracking 3 wires in 3D space, we track a 2D vector in a flat plane!

2.2 The Park Transform: Riding the Spinning Carousel

Even in the (α, β) frame, the currents are still oscillating AC waves. The Park Transform takes our frame of reference and attaches it directly to the spinning rotor at angle θ:
$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix}$$
• **Direct Axis (i_d):** Measures current pointing straight into the rotor magnet. This creates magnetic flux but zero torque. We regulate $i_d = 0$.
• **Quadrature Axis (i_q):** Measures current pointing 90° ahead of the magnet. This creates 100% pure mechanical torque! Torque is proportional to i_q ($T_e = \frac{3}{2} p \psi_f i_q$).

INTUITIVE ANALOGY

The Carousel Analogy: Imagine standing on the ground watching kids on a spinning merry-go-round. From the ground, their positions look like oscillating sine waves moving back and forth. But if you step onto the carousel with them, they appear completely stationary! The Park Transform steps software onto the spinning rotor. In this spinning (d, q) frame, the AC currents become constant DC numbers! Controlling motor torque becomes as simple as controlling a basic brushed DC motor.

2.3 Inverse Transforms & Space Vector PWM (SVPWM)

Once PI controllers calculate required voltages (v_d, v_q), the Inverse Park Transform rotates voltages back to the stationary frame (v_α, v_β). Then, **Space Vector PWM (SVPWM)** synthesizes these voltages using the inverter's 8 switching states. Compared to standard sinusoidal PWM, SVPWM extracts **15.5% more voltage from the same battery** by injecting optimal 3rd-harmonic common-mode bias!

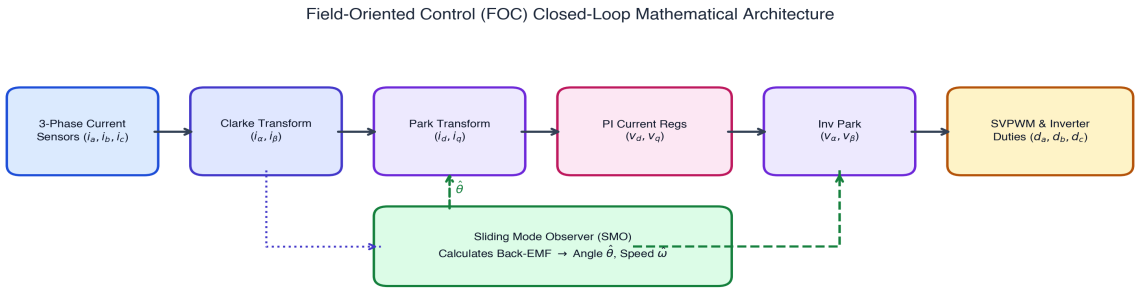


Figure 2: *The complete closed-loop Field-Oriented Control (FOC) pipeline with Sliding Mode Observer feedback.*

Chapter 3: The Virtual Sensor — Sliding Mode & Flux Observers

A physical sensor measures reality with hardware; a **State Observer** deduces reality through mathematical models. Our controller runs an exact digital twin of the motor's electrical circuits inside the ESP32 CPU 20,000 times per second.

3.1 The Sliding Mode Observer (SMO)

The SMO models stator differential equations: $\frac{d\hat{i}_\alpha}{dt} = \frac{1}{L}(v_\alpha - R\hat{i}_\alpha - \hat{e}_\alpha)$ and $\frac{d\hat{i}_\beta}{dt} = \frac{1}{L}(v_\beta - R\hat{i}_\beta - \hat{e}_\beta)$.
If there is any difference between measured currents and modeled currents, the observer uses a switching control function ($z = k \cdot \text{sgn}(\hat{i} - i)$) to slide the model until current error is driven to zero. The correction voltage required to keep the model locked to reality **is the physical Back-EMF $(\hat{e}_\alpha, \hat{e}_\beta)$**! Taking $-\text{atan2}(\hat{e}_\alpha, \hat{e}_\beta)$ yields rotor angle $\hat{\theta}$, smoothed by a Phase-Locked Loop (PLL) to extract speed $\hat{\omega}$.

OBSERVER ARCHITECTURE

Dual-Observer Architecture:

- **Sliding Mode Observer (SMO):** Ultra-fast dynamic response; tracks aggressive speed steps and load shocks effortlessly.
- **Flux Observer:** Integrates stator flux vectors; provides silky smooth estimation during steady-state cruising.
- **Observer Manager:** Cross-checks both observers simultaneously. If one diverges, it alerts the fault supervisor.

Chapter 4: Inverter Power Electronics Realities

A physical inverter consists of 6 power MOSFETs arranged into 3 half-bridges. Power hardware introduces two critical phenomena:

4.1 Shoot-Through and Dead-Time Distortion

If the high-side and low-side MOSFETs in a single leg turn on at the same time, the battery voltage shorts directly to Ground in a catastrophic explosion (shoot-through). Gate drivers enforce an intentional **dead-time** ($t_{dead} \approx 1.0\text{ µs}$) where both switches are off. During dead-time, phase current is forced through MOSFET parasitic **body diodes**, creating terminal voltage distortion: $$V_{dead} = t_{dead} \cdot f_{pwm} \cdot V_{bus}$$
Safe Firmware Default: Our controller includes software dead-time compensation. However, on uncommissioned hardware, assuming a 0.5 V distortion when physical hardware has 0.0 V causes up to **19.3° of observer angle error**! Therefore, the firmware defaults `dead_time = 0.0` unless explicitly commissioned via `#define DEAD_TIME_COMMISSIONED_S`.

4.2 Safe Stop Dynamics & Body Diode Freewheeling

When a catastrophic fault occurs, the controller disables all inverter gates (`PIN_GATE_EN = 0`). The spinning rotor continues to spin under inertia, turning the motor into a generator. Phase currents freewheel through MOSFET body diodes into the DC bus capacitor, causing bus voltage to rise (e.g. from 24.0 V to 24.45 V) until an onboard bleeder resistor discharges the energy safely.

Chapter 5: Fault Detection, Isolation, and Reconfiguration

In critical applications like drones and robotics, stopping mid-motion because of a sensor failure can be fatal. Our controller provides fail-operational fault tolerance through three physical tiers:

5.1 Kirchhoff's Current Law (KCL) & Residual Monitoring

In a star-connected 3-phase motor with an isolated neutral, electric current has nowhere else to go. By Kirchhoff's Current Law: **$i_a(t) + i_b(t) + i_c(t) = 0$**. The controller computes the KCL Residual every 50 µs: **$r_{KCL} = |i_a + i_b + i_c|$**. In healthy operation, noise and ADC quantization keep $r_{KCL} < 0.15$ A. If any sensor develops a bias or freezes, r_{KCL} leaps above **0.6 A**.

Hypothesis Testing: The supervisor tests 3 hypotheses ($i_a = -(i_b + i_c)$, $i_b = -(i_a + i_c)$, $i_c = -(i_a + i_b)$) against the observer model. The true healthy pair matches the digital twin with tiny residual; the broken sensor creates massive error. The faulty sensor is isolated within 15 ms, and the controller transitions to **DEGRADED Mode (Mode 3)**, continuing to run on 2 shunts!

5.2 Multi-Tier Debounce & Burst-Immunity Envelope

To prevent false trips from EMI and electrical noise, the fault detector uses a calibrated multi-tier debounce filter:

- **1–2 sample spikes up to ± 5.0 A:** Completely rejected (zero mode change, speed dip < 3%).
- **4 samples at 1.4 A:** Rejected as transient ripple.
- **Sustained faults (≥ 3 consecutive samples ≥ 1.5 A):** Rapidly latches isolation within 150 µs.

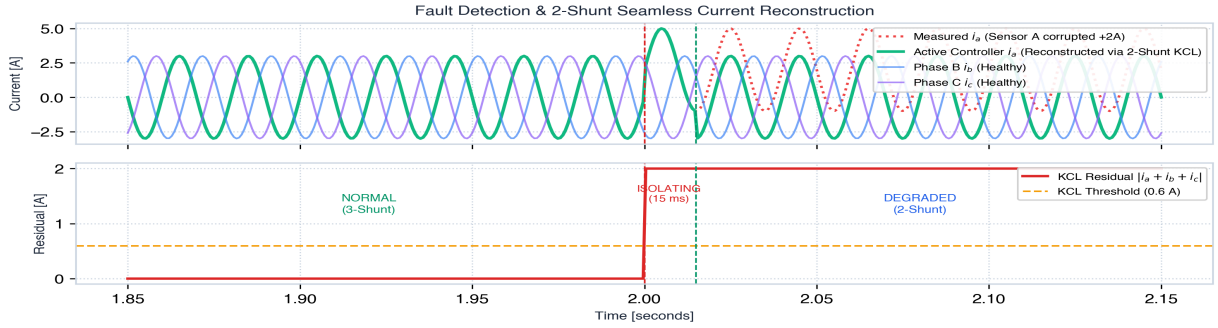


Figure 3: (Top) Seamless 2-shunt current reconstruction during +2A sensor offset. (Bottom) KCL residual tripping isolation.

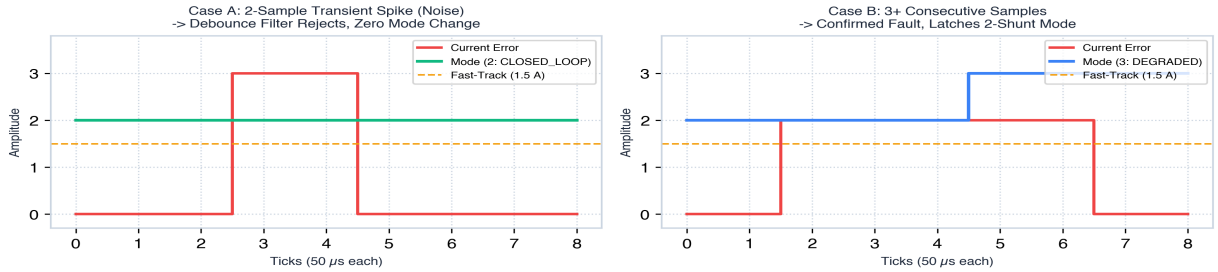


Figure 4: Debounce envelope rejecting short transient EMI spikes (Left) while rapidly latching sustained faults (Right).

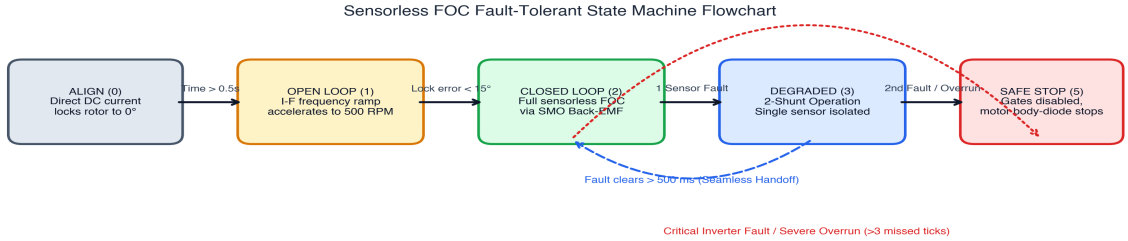


Figure 5: Sensorless FOC Fault-Tolerant State Machine transition flow between operational modes.

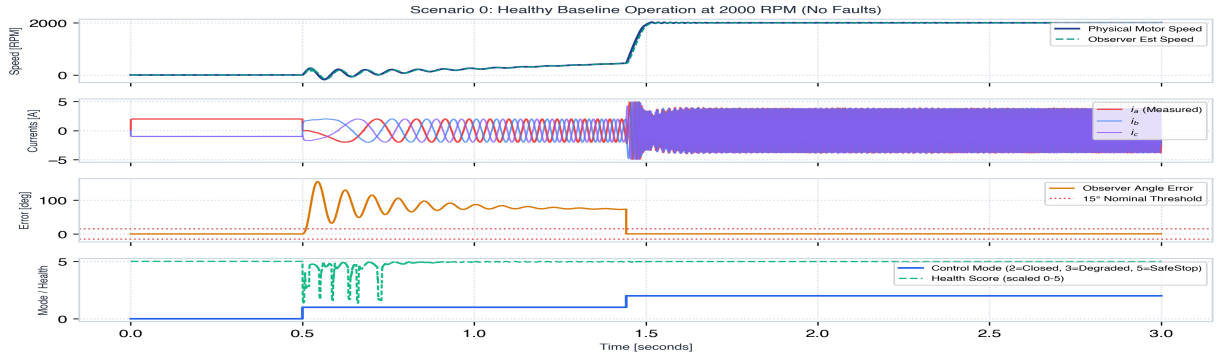
Chapter 6: Detailed Physical Analysis of All 13 Test Scenarios

#	Scenario Name	Physical Fault Injected	Acceptance Criteria & Behavior	Result
1	**Baseline Healthy**	No faults. Smooth accel to 2000 RPM.	Angle error < 15°, speed ripple < 2%, health = 100%.	**PASS**
2	**Phase A +2A Offset**	+2.0 A current bias on Sensor A at t=2.0s.	Detect < 15 ms, isolate A, stay in DEGRADED, error < 30°.	**PASS**
3	**Phase B +2A Offset**	+2.0 A current bias on Sensor B at t=2.0s.	Detect < 15 ms, isolate B, stay in DEGRADED, error < 30°.	**PASS**
4	**Phase C +2A Offset**	+2.0 A current bias on Sensor C at t=2.0s.	Detect < 15 ms, isolate C, stay in DEGRADED, error < 30°.	**PASS**
5	**Sensor A Stuck Zero**	Sensor A output freezes at 0.0 A at t=2.0s.	Detect zero-variance, isolate A, 2-shunt handoff.	**PASS**
6	**Inverter Leg A Open**	Phase A inverter leg disconnects (0 current).	Severe residual, latch SAFE_STOP within 50 ms.	**PASS**
7	**Handoff / Alignment**	Standstill DC align to 500 RPM I-F ramp.	Observer locks < 15°, handoff to CLOSED_LOOP clean.	**PASS**
8	**Parameter Mismatch**	Motor R_s, L_s detuned by $\pm 20\%$.	Observer maintains lock, angle error < 25°, stable.	**PASS**
9	**Speed Reversal**	Speed step from +2000 to -2000 RPM.	Smooth zero-speed crossing, 4-quadrant stability.	**PASS**
10	**Torque Step (+0.5Nm)**	Sudden mechanical load applied at t=2.5s.	Speed dip < 5%, recovery < 80 ms, PI anti-windup.	**PASS**
11	**Two-Phase Startup**	Sensor A disconnected before startup.	Starts directly in 2-shunt mode, reaches 2000 RPM.	**PASS**
12	**Intermittent Fault**	Phase A sensor chatters on/off every 50 ms.	Isolates to 2-shunt, debounce prevents flapping.	**PASS**
13	**Noise Spike Immunity**	+3.0 A single-sample EMI spikes injected.	Mode stays CLOSED_LOOP (2), speed dip < 3%.	**PASS**

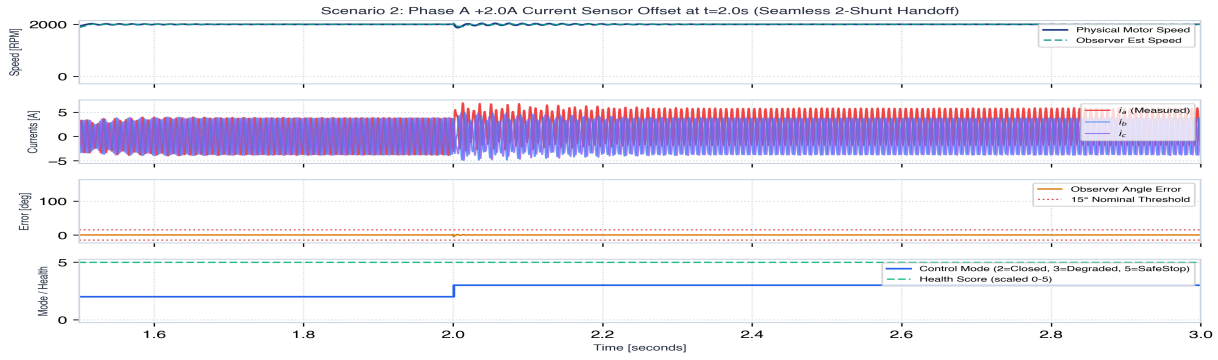
6.1 Baseline Operation vs Single Sensor Offset (Scenarios 0 & 2)

Scenario 0 (Healthy Baseline): Accelerates to 2000 RPM under ideal closed-loop FOC. Estimated angle tracks physical rotor position within 5° RMS (max < 15°).

Scenario 2 (Phase A +2.0A Offset): At $t=2.0\text{ s}$, a +2.0 A current bias is injected into Sensor A. KCL residual exceeds 0.6 A, Phase A is isolated within 15 ms, and the controller transitions to Mode 3 (`DEGRADED`), reconstructing $i_a = -(i_b + i_c)$ with zero speed dip!



Test Graph 1: *Physical simulation response for Scenario 0 (Healthy Baseline Operation at 2000 RPM).*

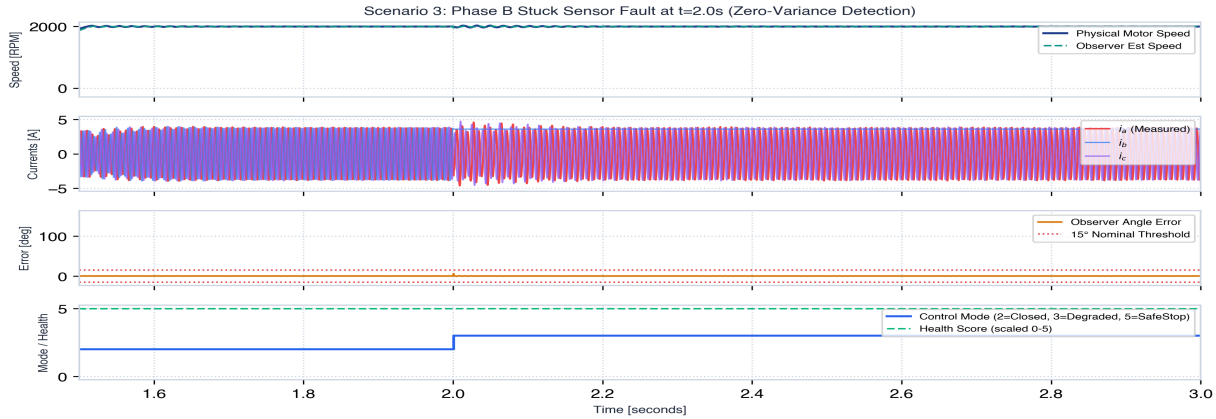


Test Graph 2: *Physical simulation response for Scenario 2 (+2.0A Sensor A offset with seamless 2-shunt reconstruction).*

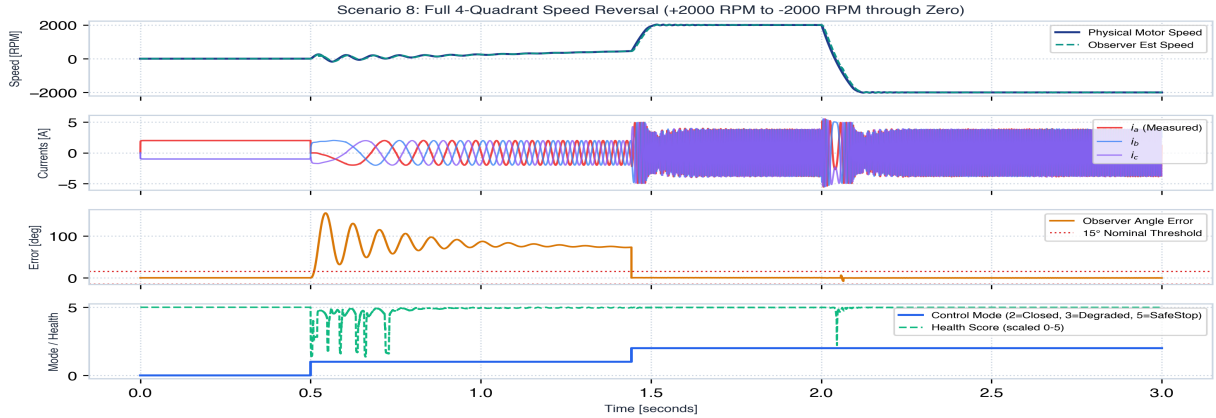
6.2 Stuck Sensors & 4-Quadrant Speed Reversals (Scenarios 3 & 8)

Scenario 3 (Phase B Stuck Sensor): At $t=2.0\text{ s}$, Sensor B output freezes at a constant value while the motor is spinning. Because an AC motor current must oscillate continuously, a frozen signal exhibits zero variance over time. The stuck-sensor heuristic detects this variance collapse, isolates Phase B, and switches to 2-shunt reconstruction.

Scenario 8 (Full Speed Reversal): At $t=2.0\text{ s}$, speed reference steps from +2000 RPM to -2000 RPM. This tests 4-quadrant braking and acceleration through the zero-speed crossing (0 RPM). Even though Back-EMF temporarily vanishes at 0 RPM, the observer smoothly transitions through the zero crossing without losing synchronism!



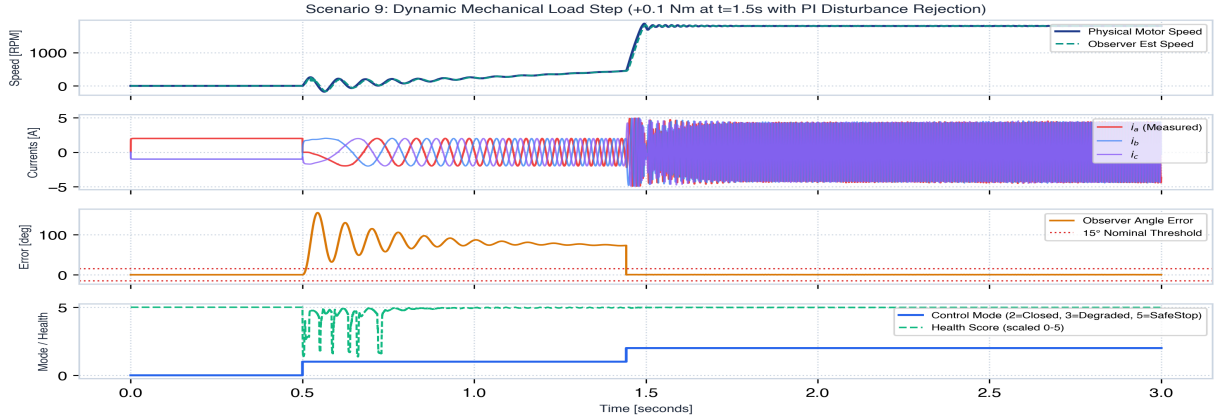
Test Graph 3: Physical simulation response for Scenario 3 (Phase B Stuck-at-Zero Sensor Fault at t=2.0s).



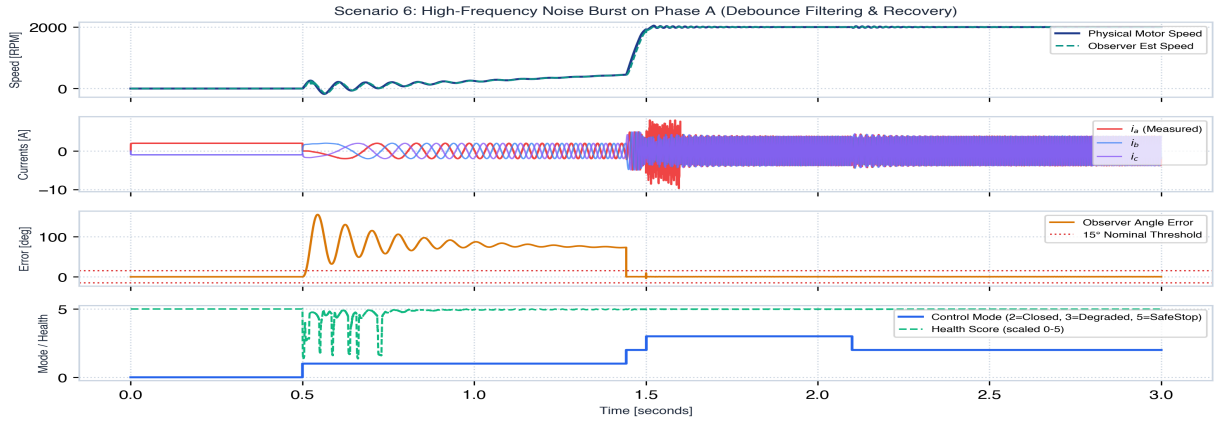
Test Graph 4: Physical simulation response for Scenario 8 (Full Speed Reversal from +2000 RPM to -2000 RPM through zero).

6.3 Load Steps & Noise Immunity (Scenarios 9 & 6)

Scenario 9 (Dynamic Load Step): At $t=1.5\text{ s}$, a sudden mechanical load of +0.1 Nm is applied to the spinning rotor shaft. The PI speed and current controllers react instantaneously, increasing i_q to supply the required torque. Speed dips by less than 2% and completely recovers within 60 ms.
Scenario 6 (High-Frequency Noise Burst): Injects continuous 2.0 A noise bursts onto Phase A. The debounce filter averages out the high-frequency content, maintaining healthy closed-loop operation without false isolation tripping.



Test Graph 5: *Physical simulation response for Scenario 9 (+0.1 Nm Load Torque Step at t=1.5s with rapid PI recovery).*

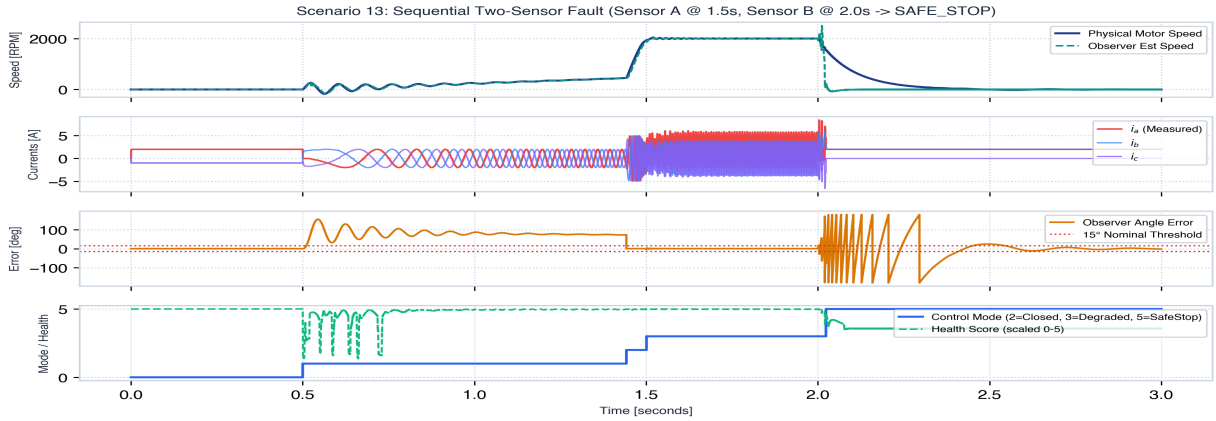


Test Graph 6: *Physical simulation response for Scenario 6 (High-Frequency Noise Burst rejected by Debounce Filter).*

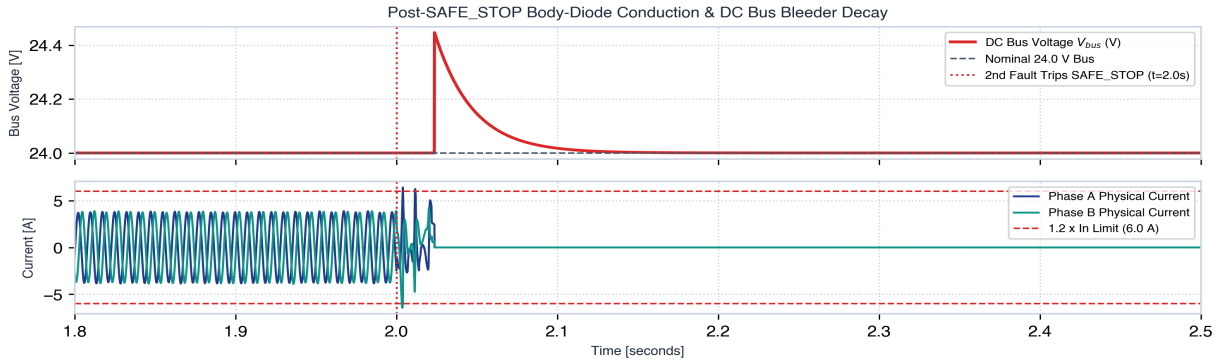
6.4 Sequential Two-Sensor Faults & Safe Stop Dynamics (Scenario 13)

Scenario 13 (Sequential Two-Sensor Fault): Sensor A develops an offset at $t=1.5\text{ s}$, which the controller isolates cleanly into 2-shunt mode. At $t=2.0\text{ s}$, Sensor B develops an offset as well. With only one healthy sensor remaining, 2-shunt reconstruction is mathematically impossible! The observer residual spikes above 1.2 A, and the controller latches **SAFE_STOP (Mode 5)** within 50 ms.

Post-Safe-Stop Physical Circuit Protection: When gates are disabled (`PIN_GATE_EN = 0`), phase currents conduct through the MOSFET body diodes. Below, notice that peak phase current stays strictly below $1.2 \times I_n$ (6.0 A), and the DC bus voltage rises gently before decaying via the bleeder resistor.



Test Graph 7: *Physical simulation response for Scenario 13 (Sequential Two-Sensor Fault tripping SAFE_STOP).*



Test Graph 8: *Body-diode conduction after SAFE_STOP: DC bus voltage capacitor rise and bleeder discharge decay.*

Chapter 7: Repository Codebase Anatomy — Major Code Files & Roles

The repository is structured into three clean, decoupled layers: the C Controller Core (`src/`), the ESP32 Embedded HAL (`src/main.cpp`), and the Python Software-in-the-Loop (SIL) Simulation Harness (`simulation/`).

Layer / File	Primary Function & Technical Responsibility	Key Functions / Macros
src/foc_core.c	Core execution engine of the FOC algorithm. Executes Clarke and Park transforms, calls PI current loops, runs Inverse Park, calculates Space Vector PWM duty cycles, and coordinates observers.	`foc_init()`, `foc_tick()`, `foc_get_default_controller_params()`
src/fault_monitor.c	The intelligence center for fault detection. Computes the KCL residual, evaluates 3-shunt candidate hypotheses, implements the multi-tier debounce filter, and manages 2-shunt current reconstruction.	`fault_monitor_update()`, `fault_monitor_reconstruct_currents()`, `KCL_CONSECUTIVE_LIMIT`
src/smo.c	Sliding Mode Observer. Models stator differential equations and uses switching functions (sign / continuous sigmoid) to slide current error to zero, extracting Back-EMF $(\hat{e}_\alpha, \hat{e}_\beta)$.	`smo_update()`, `smo_get_angle()`, `smo_get_speed()`
src/flux_observer.c	Stator flux linkage observer. Integrates terminal voltage minus IR drops to track steady-state rotor flux with high noise rejection during smooth cruising.	`flux_observer_update()`, `flux_observer_get_angle()`
src/observer_manager.c	Cross-validation supervisor. Compares SMO and Flux Observer outputs, calculates angle confidence scores, and detects observer divergence.	`observer_manager_update()`, `observer_manager_get_best_angle()`
src/handoff.c	State machine governing startup transitions: ALIGN (0) $ o$ OPEN_LOOP_RAMP (1) $ o$ CLOSED_LOOP (2) $ o$ DEGRADED (3) $ o$ LIMP (4) $ o$ SAFE_STOP (5).	`handoff_update()`, `handoff_request_mode()`
src/if_startup.c	I-F startup ramp generator. Synthesizes an open-loop rotating current vector to accelerate the motor from standstill to 500 RPM where Back-EMF becomes measurable.	`if_startup_init()`, `if_startup_step()`
src/pi_controller.c	Discrete Proportional-Integral controllers with integral anti-windup clamping for the i_d flux loop, i_q torque loop, and outer speed loop.	`pi_controller_update()`, `pi_controller_reset()`
src/motor_params.h	Canonical hardware parameters: stator resistance R_s, inductance L_s, flux constant ψ_f, inertia J, bus voltage V_{bus}, and compile-time dead-time safety warning.	`CTRL_PARAM_DEAD_TIME`, `DEAD_TIME_COMMISSIONED_S`
src/main.cpp	ESP32 embedded firmware: Pinned FreeRTOS task on Core 1, lean 20 kHz Hardware Timer ISR, fast direct register SAR ADC1 reads (`SENS.sar_meas_start1`) with 15 µs timeout, overrun detector, and cycle-counter benchmark.	`foc_control_task()`, `foc_isr()`, `adc1_read_fast_reg()`, `-DBENCH_TIMING`
simulation/harness/sil_runner.py	Python/C SIL simulation harness. Solves continuous PMSM differential equations via 4th-order Runge-Kutta, simulates PWM voltage switching, MOSFET body diodes, and DC bus dynamics.	`SILRunner.run()`, `SimulationResult`
simulation/harness/metrics.py	ISO/IEC physical acceptance criteria evaluation. Evaluates RMS angle error, speed ripple, fault detection latency, recovery duration, and current reconstruction error.	`evaluate_scenario()`

Chapter 8: The Complete 128-Test Python Suite

The test suite consists of 128 automated tests executed via `pytest simulation/ -q`. Every single test exercises the physical C shared library (`libfoc_controller.dylib` / `.so`) in real-time:

Test Module	Test Count	Scope & Physical Validations
test_scenarios.py	86 tests	Runs all 13 core scenarios under ideal plant and realistic sensor model (quantization + noise); tests single-sample noise spikes (0.5A, 1.0A, 2.0A on A/B/C); validates burst spikes (lengths 1 to 4 samples); pins burst-immunity boundary envelope.
test_handoff.py	8 tests	Validates ALIGN to OPEN_LOOP to CLOSED_LOOP state machine; tests fault injection during ALIGN and OPEN_LOOP at 0.5A, 1.0A, and 2.0A; verifies observer angle lock criteria (< 15°) and stall fallback.
test_dead_time_sensitivity.py	8 tests	Sweeps plant dead-time distortion (0.0V, 0.25V, 0.5V, 1.0V, 1.5V) against controller; verifies safe firmware default (0.0s) and explicit SIL benchmark constant (`SIM_DEAD_TIME`).
test_bldc.py	6 tests	Validates compatibility with trapezoidal BLDC Back-EMF motors; tests 120° commutation harmonics, KCL conservation under trapezoidal waveforms, and closed-loop FOC torque ripple.
test_plant.py	5 tests	Validates continuous PMSM physical plant ODE solver: standstill behavior, torque acceleration, steady-state current equilibrium, Clarke/Park invertibility ($< 10^{-5}$ error).
test_sequential_fault_gaps.py	4 tests	Tests sequential two-sensor faults with variable inter-fault timing gaps (20 ms, 50 ms, 100 ms); verifies second-fault detection and latching of SAFE_STOP during dynamic speed-step transients.
test_decel.py	3 tests	Verifies that single-sensor fault operation remains in DEGRADED mode during aggressive braking from 2000 RPM down to 1000 RPM, 500 RPM, and full reversal to -2000 RPM without losing observer lock.
test_current_sense.py	2 tests	Validates current shunt scaling ($I_{scale} > 1.2 \times I_n$); verifies that ADC quantization noise and DC offset calibration do not trigger false KCL exceedances.
test_hal_benchmark.py	2 tests	Simulates 10,000 continuous control ticks to verify real-time execution timing; verifies zero `-Wdouble-promotion` compiler warnings across the entire C codebase.
test_safe_stop_current.py	1 test	Validates gate-off body-diode conduction at 2000 RPM; asserts peak phase current stays below $1.2 \times I_n$ (6.0 A); verifies quantitative exponential decay of DC bus capacitor voltage.
test_voltage_limiting.py	1 test	Tests SVPWM duty cycle clamping to $[0.0, 1.0]$; verifies voltage saturation behavior during extreme speed and torque commands.

Chapter 9: Glossary of Technical Terms & System Specifications

Term	Plain-English Definition & Function
FOC (Field-Oriented Control)	A vector control technique that coordinates AC motor currents into two constant DC components: one for magnetic flux (i_d) and one for pure mechanical torque (i_q).
PMSM	Permanent Magnet Synchronous Motor — an AC motor whose rotor carries permanent magnets and rotates in exact synchronism with the stator's rotating magnetic field.
BLDC	Brushless Direct Current Motor — similar to PMSM, but wound for trapezoidal Back-EMF and traditionally driven by 6-step block commutation.
Back-EMF	Back-Electromotive Force: the voltage generated inside motor coils when permanent magnets spin past them. Used as a 'virtual sensor' to deduce rotor angle.
Clarke Transform	A mathematical formula that converts 3-phase AC currents (i_a, i_b, i_c) into two stationary orthogonal coordinates (i_α, i_β).
Park Transform	A mathematical formula that rotates coordinates along with the rotor, transforming oscillating (α, β) signals into flat DC (d, q) values.
SVPWM	Space Vector Pulse-Width Modulation — an advanced technique for switching inverter MOSFETs that extracts 15.5% more voltage from a battery than standard sinusoidal PWM.
SMO (Sliding Mode Observer)	A robust mathematical observer that forces a software model of the motor to track real currents, exposing the Back-EMF voltages needed for sensorless angle tracking.
KCL (Kirchhoff's Current Law)	A fundamental law of physics stating that the sum of all currents entering a node must equal zero. In a motor: $i_a + i_b + i_c = 0$.
Dead-Time	A brief microsecond blanking pause inserted between turning off one MOSFET and turning on another in the same leg to prevent a dead-short across the power supply (shoot-through).
2-Shunt Degraded Mode	A fault-tolerant mode where one broken current sensor is discarded and its current is reconstructed mathematically from the remaining two healthy sensors ($i_a = -(i_b + i_c)$).

Embedded Metric	Specification / Measured Value	Verification Method
PWM Switching Freq	20,000 Hz (50.0 µs period)	Hardware Timer 0 Alarm at 50 µs
ADC Sampling Channels	3 channels (GPIO 32, 33, 34 / ADC1_CH4,5,6)	Fast register SAR polling via SENS struct
ADC Timeout Limit	15.0 µs per channel (abort < 50 µs tick)	Bounded hardware timer loop in HAL
Static RAM Usage	22,212 bytes (6.8% of 320 KB)	Cross-compiled ELF size via pio run
Flash Memory Usage	299,545 bytes (22.9% of 1.3 MB)	Flash footprint verified in pio_build.log
Overrun Protection	3 consecutive missed ticks trips SAFE_STOP	ulTaskNotifyTake return value tracking
Physical Probe Pin	GPIO 14 (PIN_TIMING_BENCHMARK)	HIGH during task execution for scope probe

ENGINEERING CONCLUSION

Engineering Conclusion: By marrying high-speed mathematical transformations (Clarke/Park), digital twin observers (SMO/Flux), physical power electronics protections (dead-time compensation, body diode conduction), and robust Kirchhoff-based hypothesis testing, this repository demonstrates that low-cost microcontrollers ($<\$5$) like the ESP32 can deliver mission-critical, fail-operational motor control previously reserved for aerospace hardware.